Electrical Fault Detection

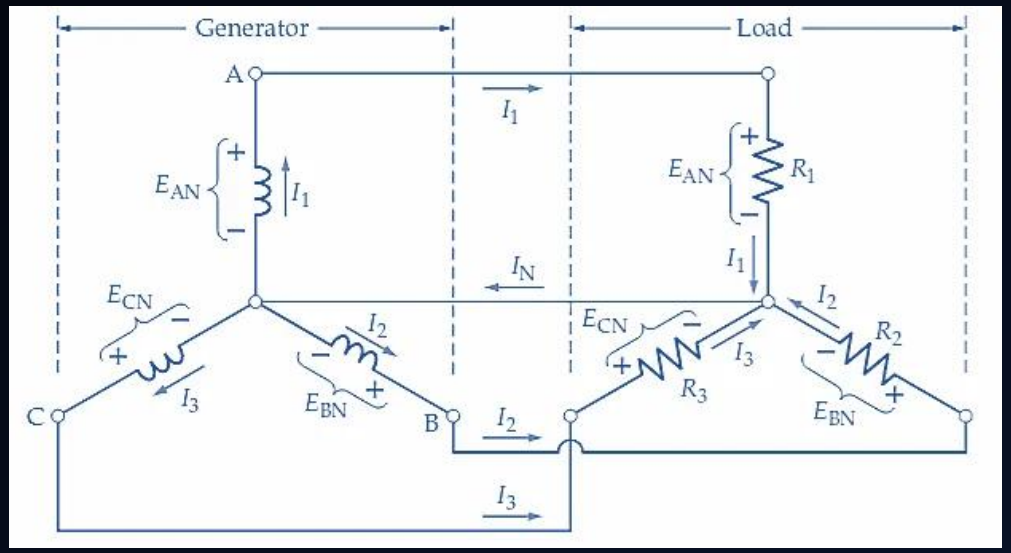
CS4320 CAPSTONE – LEVI DOCKSTADER

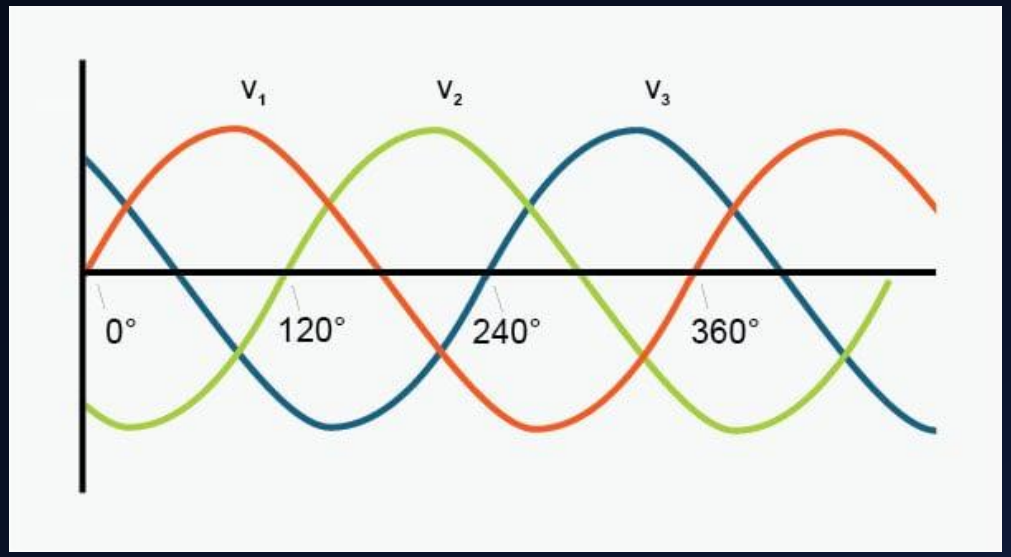
PROBLEM STATEMENT

- DEFINITION: USE MACHINE LEARNING TO DETECT AND CLASSIFY ELECTRICAL FAULTS IN A SIMULATED 3-PHASE GRID SYSTEM USING ONLY VOLTAGE AND CURRENT VALUES
- TASK TYPE: MULTICLASS CLASSIFICATION, THOUGH ORIGINALLY BINARY WAS USED
- APPLICATION: SUPPORT DECISIONS ABOUT GRID MANAGEMENT BY HELPING PREDICT AND DETECT ISSUES USING NON-TRANSIENT DATA

Background: 3 Phase Circuits

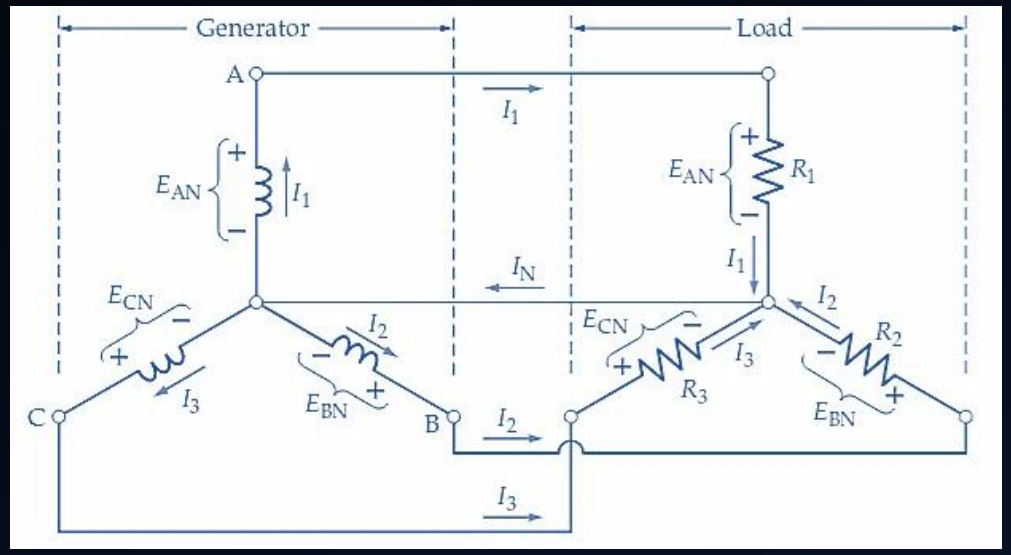
- 3 phase – 3 sine waves (ABC) offset from the others by 120°
- Ground – Common reference shared by all phases
- Ohm's Law: V = IZ
  - V: Voltage (V)
  - I: Current (A)
  - Z: Impedance (Ω)

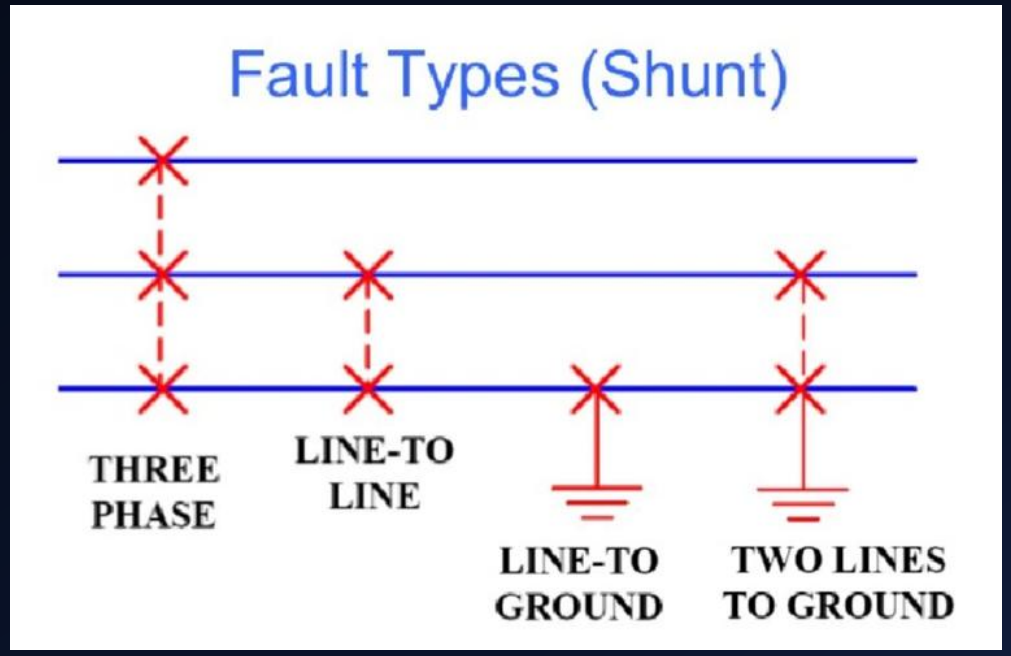




Background: Grid Faults

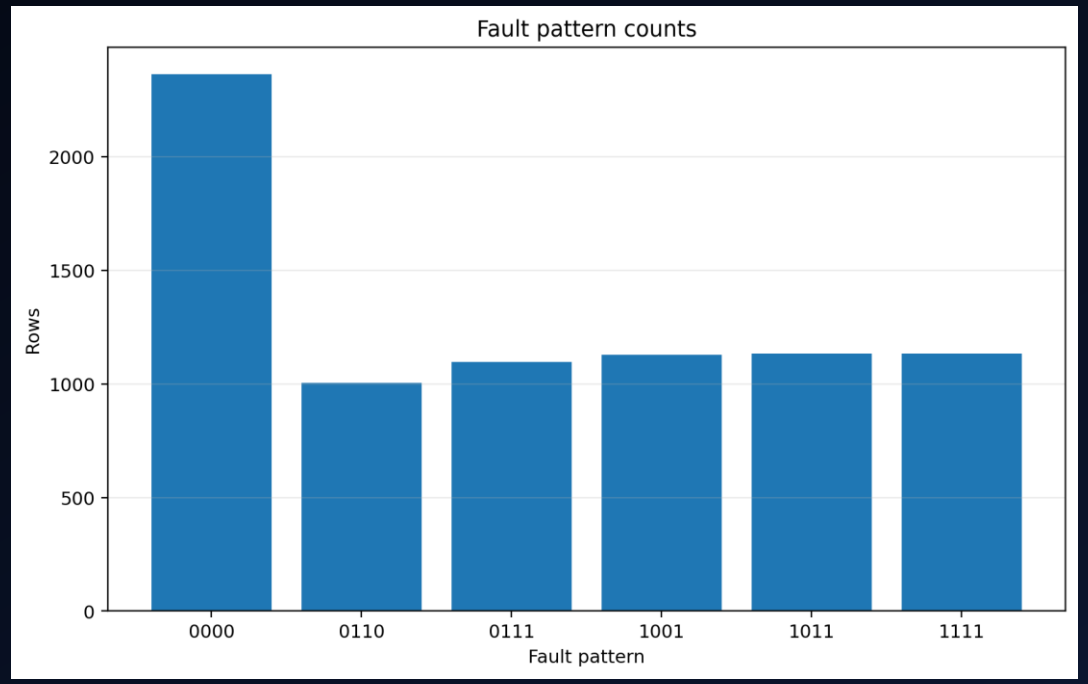
- During a fault, a line is somehow damaged, causing it to either touch another line or the ground.
- Faults cause electrical values to change from what's expected.
  - Typically, voltage and impedance decrease while current increases.



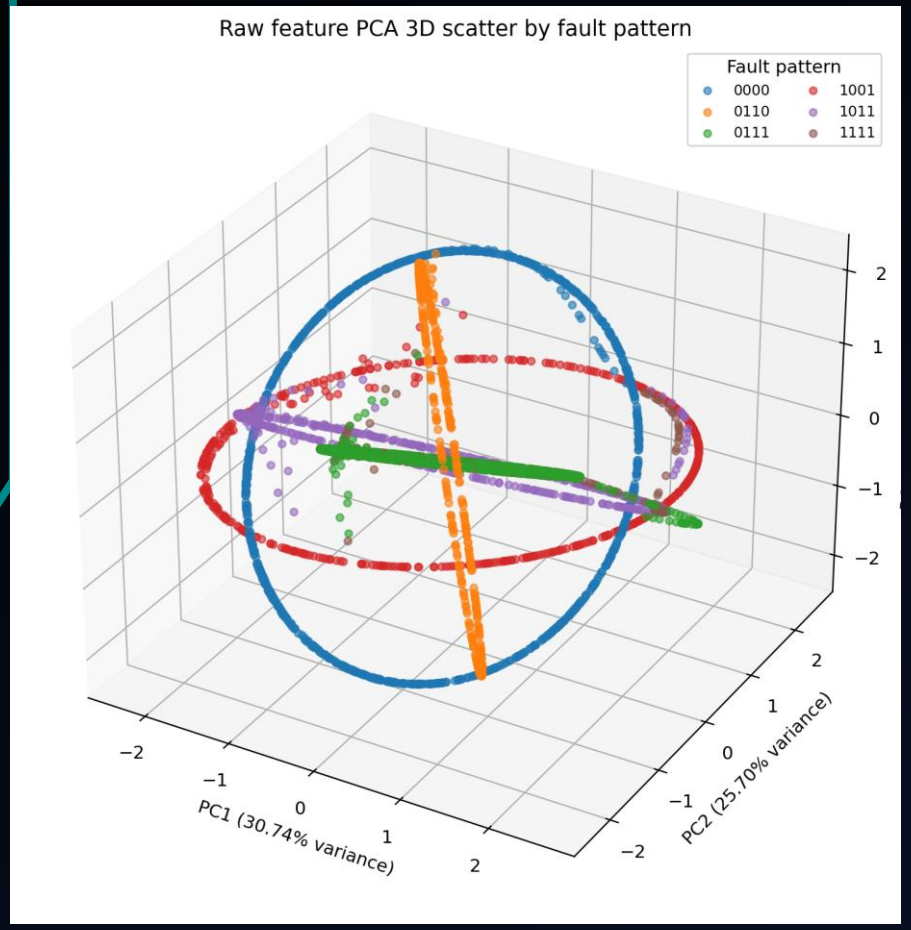


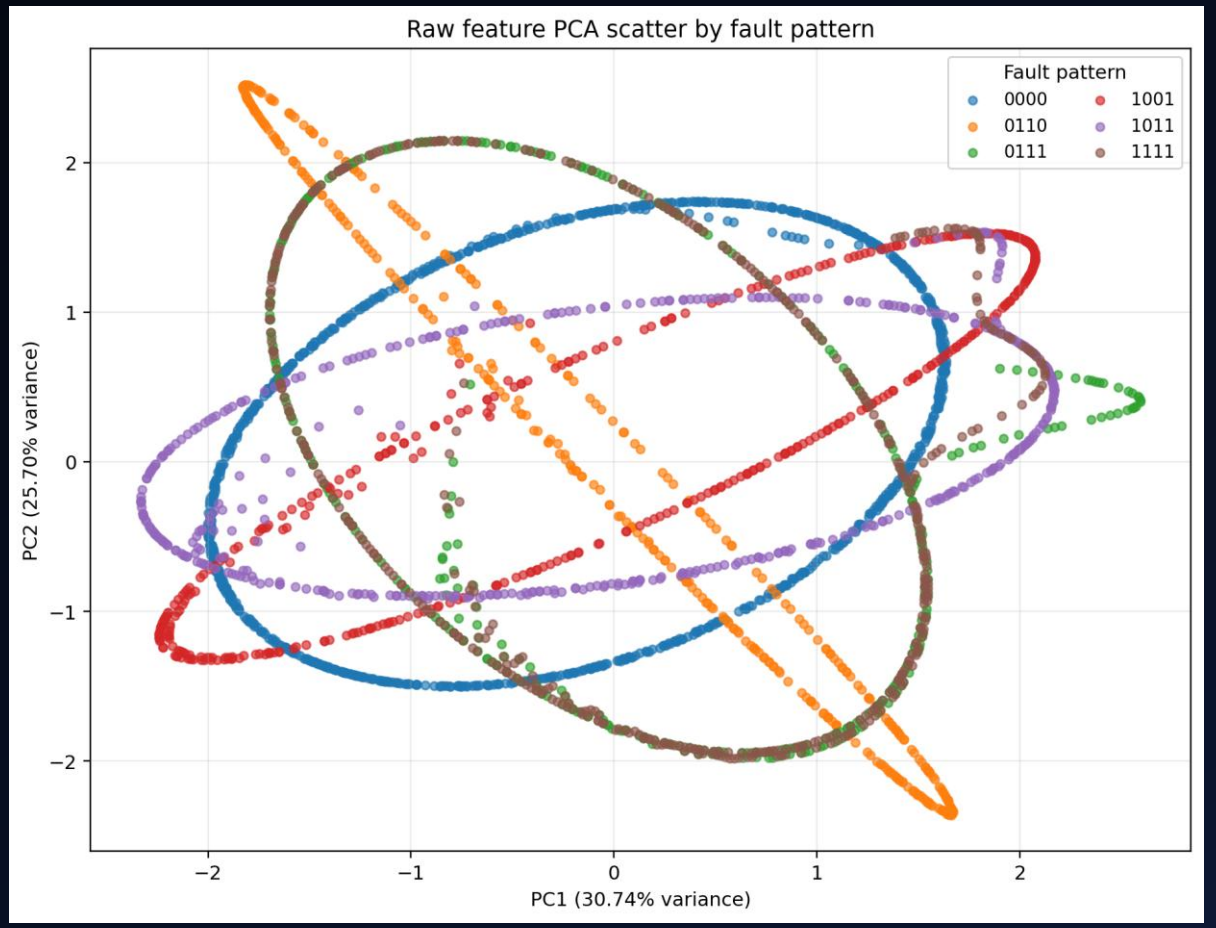
APPROACH AND ARCHITECTURE

- Input:
  - 6 raw features
    - 3 voltage values
    - 3 current values
    - Labels excluded to prevent leakage
  - 7681 rows
- Output:
  - 4-digit-binary fault classifier prediction
    - GCBA 0000-1111
  - Early experiments performed binary fault classification
- Feature engineering:
  - 10 proxy features
    - Each proxy is related to a data point's impedance and phase.
    - Proxies were only developed during my final model experimenting.

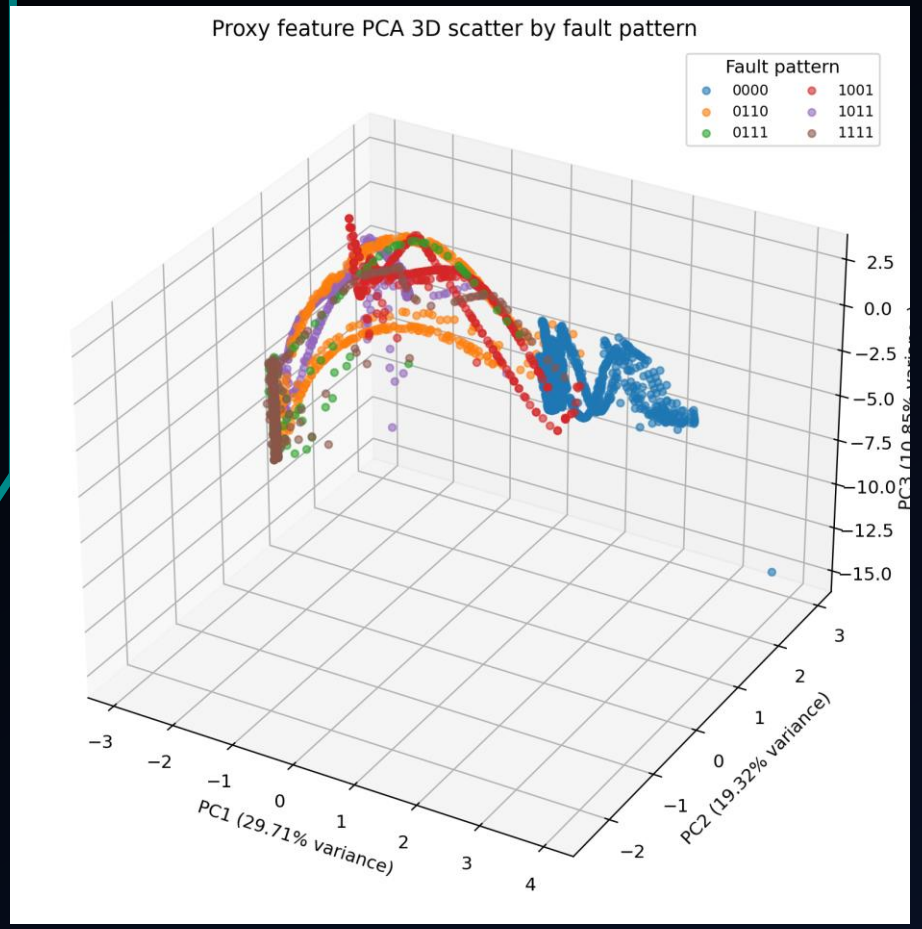


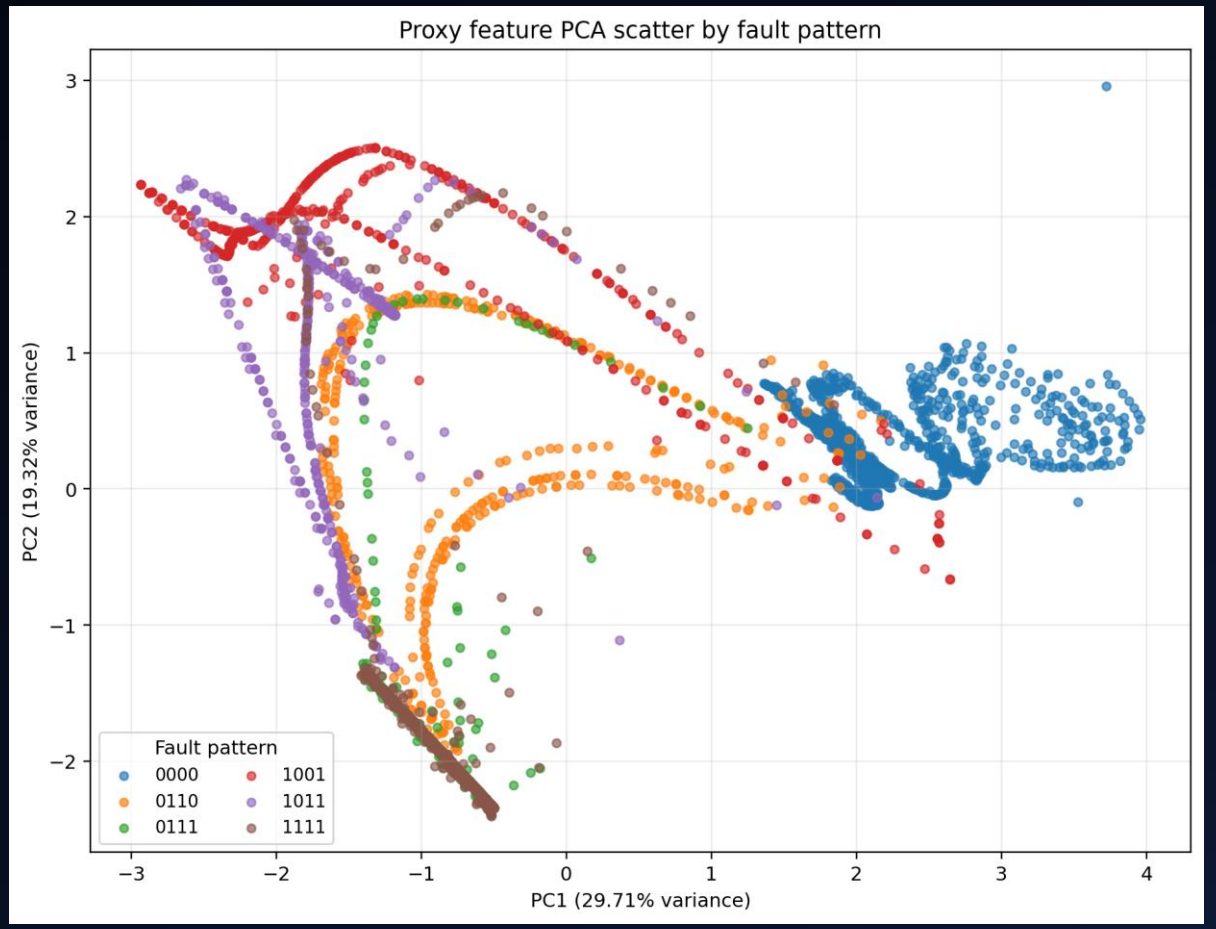
Searching for Structure – Raw Feature Visualization





Searching for Structure – Proxy Feature Visualization





MODEL SELECTION

Key considerations

- Relatively small dataset
- Tabular
- Non-linear
- Data appears clean and low-noise
- Simulated circuit data
- Mild class imbalance

- Trial models included top homework performers.
  - kNN
  - RBF SVM
  - Random Forest
  - MLP
- Without proxies, kNN performed best.
  - accuracy=0.8887
  - balanced_accuracy=0.8691,
  - f1=0.8691
- With proxies, **Random Forest** performed excellently, making it the final model choice.

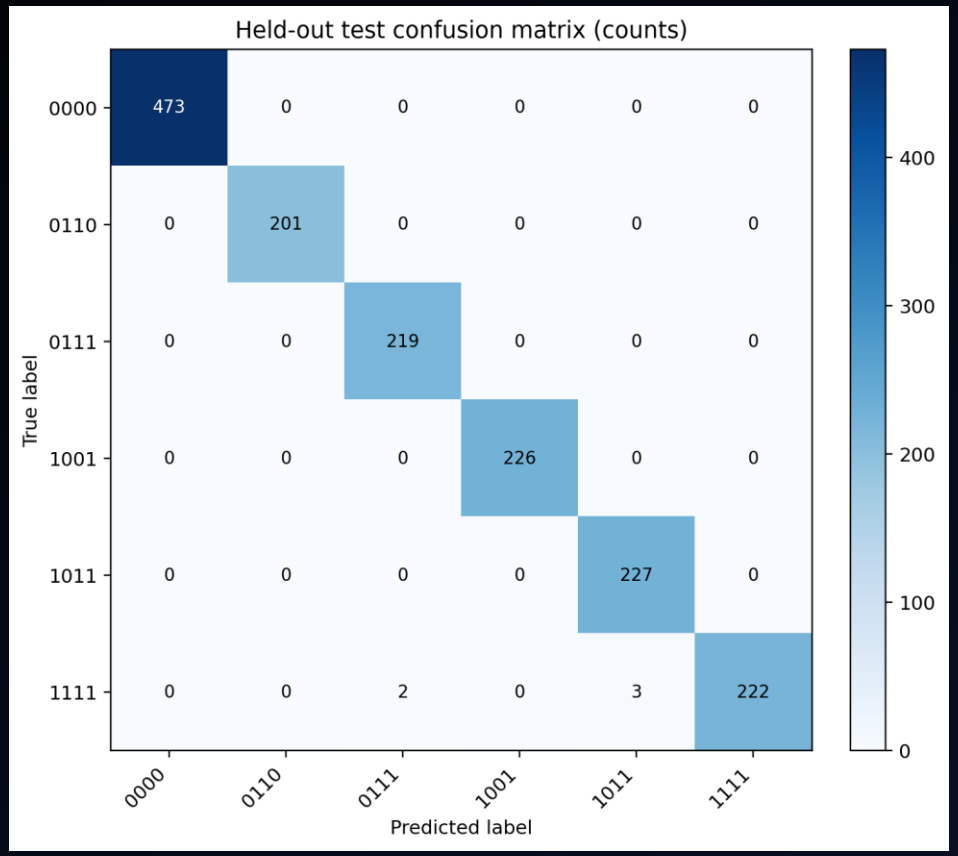
Capstone Code Workflow

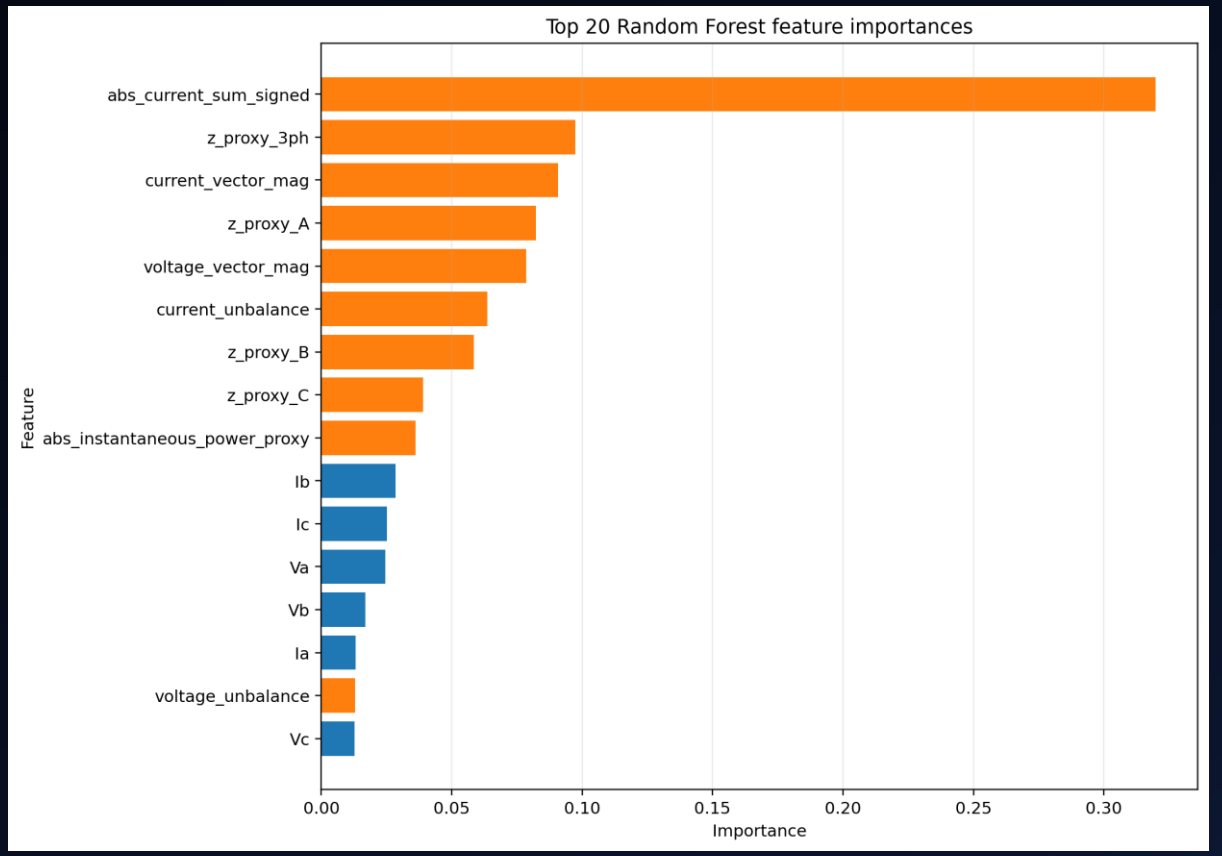
1. Load and preprocess fault data from a csv, including building proxy features.
2. Compare three feature sets: raw only, proxy only, and raw plus proxy.
3. Split the data into training, validation, and test sets in a leakage-safe way.
4. Use the training and validation sets to find the best feature representation.
5. Tune a Random Forest on the best feature set using validation balanced accuracy.
6. Retrain the best model on all non-test data.
7. Evaluate it once on the held-out test set.
8. Save the final outputs, including metrics, confusion matrices, feature importance, predictions, and the trained model file.

RESULTS AND OBSERVATIONS

- Model training highlights:
  - Tuned with a grid search
  - 300 trees
  - Mean depth: 15
  - Mean leaf count: 112
  - Balanced subsample class weight
  - OOB score: 99.7%
  - Perfect training metrics
  - 99.7% validation accuracy.
- Test performance:
  - Accuracy: 99.68%
  - Balanced accuracy: 99.63%
  - Precision: 99.63%
  - Recall: 99.63%
  - F1: 99.63%
  - Only 5 wrong predictions

Model Performance Visuals





INTERPRETATION

- Feature representation matters significantly.
- Having real-world understanding helps motivate useful decisions.
- Higher complexity does not necessarily mean superior performance
  - Random forest beat MLP.
- Visualizing feature-space geometry helps inform transformation decisions.
- Multiple models can perform well on the same task.
  - Other factors should also influence final decision.
- Validation and test results show that random forest was a good choice for this problem.

CONCLUSION

- Main takeaway: spend more time thinking about the data and problem to make good model choices before spending a lot of effort tuning a model that was a sup-par decision.
- While real-time sensors are actually used to detect faults, applying machine learning techniques still revealed behavioral patterns and performance structure that is useful in understanding fault detection.
- Sometimes decision trees outperform neural networks.